Métodos de Optimización

Clase 2: Métodos de búsqueda determinísticos y básicos

[Nombre del profesor]

Septiembre de 2026

Programa de Maestría, [Nombre de la universidad / programa]

Contenido

1. Motivación: ¿por qué buscar si ya vimos condiciones analíticas?
2. Bisección en 1D
3. Búsqueda áurea en 1D
4. Bisección vs. áurea
5. Búsqueda exhaustiva (grid search) en 1, 2 y 3 variables
6. Random walk en 2D
7. Nelder–Mead (amoeba) en 2D
8. Síntesis y tabla comparativa
9. Vista previa: Clase 3
10. Ejercicios y referencias

Motivación

¿Por qué necesitamos métodos de búsqueda?

En la Clase 1 vimos condiciones **analíticas**: resolver $\nabla f(\mathbf{x}) = \mathbf{0}$, clasificar con la Hessiana, usar Lagrange/KKT. Todo eso requiere que f sea **diferenciable** y que podamos **resolver el sistema de ecuaciones** resultante.

¿Qué pasa si...?

- no conocemos una fórmula cerrada para f (es una simulación, un experimento, una caja negra)?
- f no es diferenciable, o su derivada es muy costosa/imposible de calcular?
- $\nabla f(\mathbf{x}) = \mathbf{0}$ no se puede resolver algebraicamente?
- f tiene muchísimos mínimos locales y no confiamos en que el descenso de gradiente encuentre el bueno?

Hoy: métodos que solo necesitan **evaluar** f (a veces f'), sin resolver ecuaciones. Empezamos por los más simples y deterministas.

El menú de hoy

1 variable

- Bisección (usa f')
- Búsqueda áurea (no usa f')

Cualquier número de variables

- Búsqueda exhaustiva (grid search)

2 (o más) variables, sin derivadas

- Random walk
- Nelder–Mead (amoeba)

Todos son **deterministas** o casi (random walk tiene una parte aleatoria, pero es el más simple posible). En la **Clase 3** veremos metaheurísticas más sofisticadas que combinan aleatoriedad con reoptimización: basin hopping, simulated annealing, algoritmos genéticos y particle swarm.

Cada método se muestra con **código manual** (para entender el mecanismo) y su **equivalente en scipy** (para comparar contra una implementación profesional).

Bisección en 1D

Bisección para minimizar: la idea

Idea

Un mínimo de f diferenciable es una **raíz de** f'. Si tenemos $[a, b]$ con $f'(a)$ y $f'(b)$ de signos opuestos (uno negativo, uno positivo), sabemos que hay un mínimo adentro — y podemos usar **bisección clásica sobre** f' para encontrarlo.

Algoritmo

1. $c = (a + b)/2$; evaluar $f'(c)$.
2. Si $f'(c)$ tiene el mismo signo que $f'(a)$: $a \leftarrow c$. Si no: $b \leftarrow c$.
3. Repetir hasta que $b - a$ sea suficientemente chico.

Convergencia

Lineal: el intervalo se reduce a la **mitad** en cada iteración, garantizado, sin importar qué tan fea sea f (mientras cumpla la condición de signos). Necesita poder evaluar f'.

Bisección: ejemplo paso a paso

$f(x) = x^2 - 2x + 3$, $f'(x) = 2x - 2$ (la misma función de la Clase 1, mínimo exacto en $x^* = 1$). Intervalo inicial $[-2, 5]$: $f'(-2) = -6 < 0$, $f'(5) = 8 > 0$. Primeras iteraciones reales:

```
it= 1  a=-2.000000  b= 5.000000  c= 1.500000  f'(c)= 1.000000  ancho=7.000000
it= 2  a=-2.000000  b= 1.500000  c=-0.250000  f'(c)=-2.500000  ancho=3.500000
it= 3  a=-0.250000  b= 1.500000  c= 0.625000  f'(c)=-0.750000  ancho=1.750000
it= 4  a= 0.625000  b= 1.500000  c= 1.062500  f'(c)= 0.125000  ancho=0.875000
it= 5  a= 0.625000  b= 1.062500  c= 0.843750  f'(c)=-0.312500  ancho=0.437500
      ...
it=22  ancho=0.0000033  ->  x* = 1.000000   f(x*) = 2.000000
```

22 iteraciones para reducir el intervalo a $3{,}3 \times 10^{-6}$ — cada iteración exactamente a la mitad de la anterior: $7 \times 0{,}5^{22} \approx 1{,}7 \times 10^{-6}$, consistente con lo observado.

Bisección: código

Manual:

```
def biseccion_min(fprime, a, b, tol=1e-6):
    while True:
        c = 0.5*(a + b)
        fc = fprime(c)
        if abs(fc) < tol or (b-a) < tol:
            return c
        if np.sign(fc) == np.sign(fprime(a))
            :
            a = c
        else:
            b = c
```

Con scipy:

```
from scipy.optimize import brentq

x_star = brentq(fprime, a, b, xtol=1e-6)
```

`brentq` resuelve el mismo problema (raíz de f') pero combina secante e interpolación cuadrática inversa con bisección — converge **mucho** más rápido que bisección pura, garantizando robustez.

Ambos dan $x^* = 1{,}000000$, $f(x^*) = 2{,}000000$. Script completo: `scripts/01_biseccion_1d.py`

Búsqueda áurea en 1D

Búsqueda áurea: la idea

Idea

A diferencia de bisección, **no usa** f' — solo evalúa f. Sirve para cualquier función **unimodal** (un solo mínimo) en $[a, b]$, incluso si no es diferenciable.

Algoritmo

Mantiene $[a, b]$ con dos puntos interiores $x_1 < x_2$ colocados con la razón áurea $\varphi = \frac{\sqrt{5}-1}{2} \approx 0{,}618$:

- Si $f(x_1) < f(x_2)$: el mínimo está en $[a, x_2]$ — descartamos la derecha.
- Si $f(x_1) > f(x_2)$: el mínimo está en $[x_1, b]$ — descartamos la izquierda.

La razón áurea no es casualidad

Con esa proporción, uno de los dos puntos interiores **se reutiliza** de la iteración anterior — cada paso necesita solo **una** evaluación nueva de f, no dos.

Búsqueda áurea: ejemplo paso a paso

Mismo problema: $f(x) = x^2 - 2x + 3$ en $[-2, 5]$.

```
it= 1  a=-2.00  b= 5.00  x1= 0.674 x2= 2.326  f(x1)=2.106  f(x2)=3.759  ancho=7.000
it= 2  a=-2.00  b= 2.33  x1=-0.348 x2= 0.674  f(x1)=3.816  f(x2)=2.106  ancho=4.326
it= 3  a=-0.35  b= 2.33  x1= 0.674 x2= 1.305  f(x1)=2.106  f(x2)=2.093  ancho=2.674
it= 4  a= 0.67  b= 2.33  x1= 1.305 x2= 1.695  f(x1)=2.093  f(x2)=2.483  ancho=1.652
      ...
it=34  ancho=0.00000089  ->  x* = 1.000000   f(x*) = 2.000000
```

34 iteraciones (vs. 22 de bisección) para la misma tolerancia: áurea reduce el intervalo por un factor $\varphi \approx 0{,}618$ por iteración (más lento que el 0,5 de bisección), pero a cambio **no necesita derivada**.

Búsqueda áurea: código

Manual (núcleo del bucle):

```
PHI = (5**0.5 - 1) / 2   # ~0.618
x1 = b - PHI*(b-a); x2 = a + PHI*(b-a)
f1, f2 = f(x1), f(x2)
while (b - a) > tol:
    if f1 < f2:
        b = x2; x2, f2 = x1, f1
        x1 = b - PHI*(b-a); f1 = f(x1)
    else:
        a = x1; x1, f1 = x2, f2
        x2 = a + PHI*(b-a); f2 = f(x2)
```

Con scipy:

```
from scipy.optimize import
     minimize_scalar

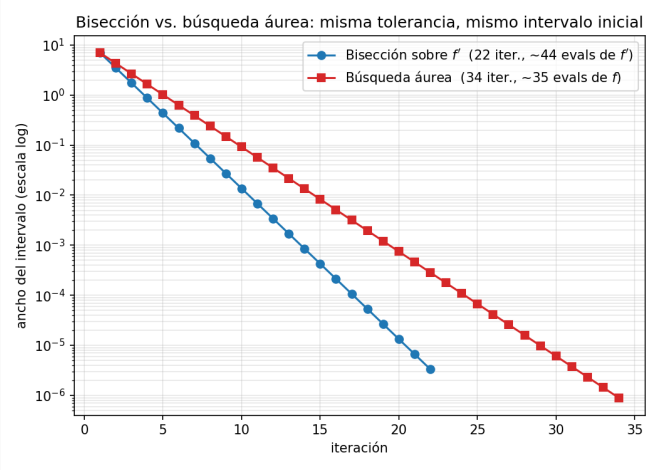
res = minimize_scalar(
    f, bracket=(a, (a+b)/2, b),
    method="golden")
```

scipy implementa exactamente este algoritmo con ese nombre.

Script completo: `scripts/02_busqueda_aurea_1d.py`

Bisección vs. áurea

Bisección vs. búsqueda áurea



¿Cuál usar?

- Si f' está disponible y es barata de evaluar: **bisección** (o mejor, un método tipo Brent que combina varias ideas) converge con menos iteraciones.
- Si f no es diferenciable, o f' es costosa/no disponible: **búsqueda áurea** es la opción segura — solo necesita evaluar f.
- Ambos requieren un intervalo inicial $[a, b]$ donde se sepa que hay un único mínimo (unimodalidad). Sin esa garantía, ninguno de los dos promete encontrar el mínimo **global**.

Conexión con la Clase 1

Ambos son primos de los métodos de la Clase 1: bisección usa información de primer orden (el signo de f', como la condición $\nabla f = \mathbf{0}$); áurea es puramente .^{ev}aluar y comparar", la filosofía que vamos a extender hoy a varias variables.

Búsqueda exhaustiva (grid search)

Grid search: la idea

Idea

Evaluar f en **todos los puntos de una malla regular** y quedarse con el mejor. No requiere derivadas, no requiere que f sea continua, **no le importa cuántos mínimos locales tenga** f — siempre encuentra el mínimo *de la malla*, que se acerca al mínimo real si la malla es suficientemente fina.

El precio: la maldición de la dimensionalidad

Con N puntos por eje y d variables, el costo es N^d evaluaciones. Con $N = 50$:

$$d{=}1 \to 50 \qquad d{=}2 \to 2{,}500 \qquad d{=}3 \to 125{,}000 \qquad d{=}4 \to 6{,}250{,}000 \ \ldots$$

Por eso grid search es práctico en 1–3 variables, y rápidamente inviable más allá.

Grid search en 1, 2 y 3 variables: código

```
# 1D: N evaluaciones
xs = np.linspace(-2, 5, N);  i = np.argmin(f1d(xs))

# 2D: N^2 evaluaciones
X, Y = np.meshgrid(np.linspace(-10,10,N), np.linspace(-10,10,N))
Z = f2d(X, Y);  idx = np.unravel_index(np.argmin(Z), Z.shape)

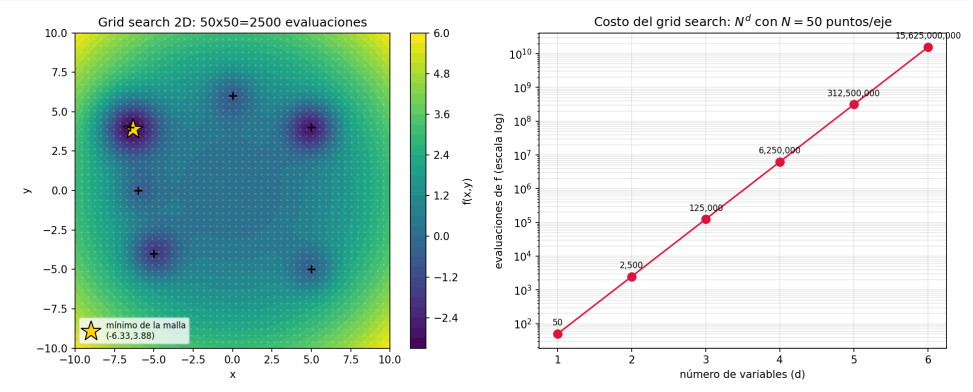
# 3D (o dD en general): N^d evaluaciones, con itertools.product
import itertools
mejor = min(itertools.product(xs3, ys3, zs3), key=lambda p: f3d(*p))
```

Con scipy:

```
from scipy.optimize import brute
res = brute(f2d, ranges, finish=None)        # grid puro
res = brute(f2d, ranges)                     # + pule con Nelder-Mead (default)
```

Script completo: `scripts/04_grid_search_1_2_3d.py`

Grid search: resultado y costo



Izquierda: malla 50×50 sobre un paisaje con 6 mínimos locales — grid search encuentra el global sin ningún truco. Derecha: el costo N^d crece brutalmente con la dimensión.

Grid search: ventajas y límites

A favor

- Encuentra el mínimo **global** de la malla, garantizado.
- No necesita nada sobre f: ni continuidad, ni convexidad, ni derivadas.
- Trivialmente paralelizable (cada evaluación es independiente).

En contra

- Costo N^d: inviable en más de 3–4 variables.
- La precisión está limitada por qué tan fina sea la malla.
- Desperdicia evaluaciones en zonas obviamente malas.

Idea que retomaremos

Los métodos que siguen (random walk hoy; basin hopping, simulated annealing, GA, PSO en la Clase 3) tratan de **gastar las evaluaciones de forma más inteligente** que una malla uniforme.

Random walk en 2D

Random walk: la idea

Distinto de grid search y de Monte Carlo puro

Un **random walk** da un paso aleatorio **a partir del punto actual** — el recorrido es una cadena donde cada punto depende del anterior (a diferencia de grid search, que es una malla fija, y de random search/Monte Carlo, que muestrea puntos independientes en todo el dominio).

Dos variantes:

- **Greedy**: el paso solo se acepta si mejora f; si no, se descarta y se prueba otro paso aleatorio desde el mismo punto.
- **Puro**: el paso **siempre** se acepta, mejore o no — pura difusión, no converge a nada, solo explora.

Sin quererlo, ya estamos a un paso de simulated annealing (Clase 3): si en vez de "siempre.º "solo si mejoraüsamos una probabilidad tipo Metrópolis, ahí aparece.

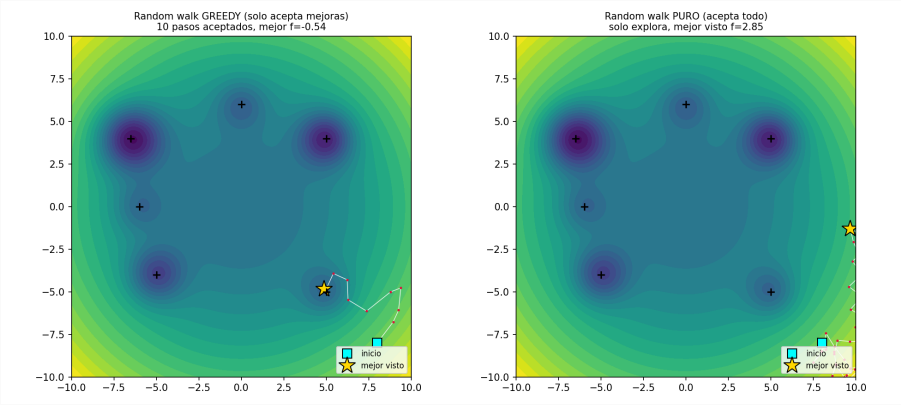
Random walk: código

```python
def random_walk_greedy(f, x0, step=1.5, n_iter=500):
    x = np.array(x0, float);  fx = f(*x)
    for _ in range(n_iter):
        x_new = x + rng.uniform(-step, step, size=2)
        f_new = f(*x_new)
        if f_new < fx:                  # solo se mueve si MEJORA
            x, fx = x_new, f_new
    return x, fx

def random_walk_puro(f, x0, step=1.5, n_iter=500):
    x = np.array(x0, float);  x_best, f_best = x.copy(), f(*x)
    for _ in range(n_iter):
        x = x + rng.uniform(-step, step, size=2)   # SIEMPRE se mueve
        if f(*x) < f_best: x_best, f_best = x.copy(), f(*x)
    return x_best, f_best
```

No existe una función de scipy llamada random walk"(es demasiado simple para uso general). Como comparación honesta usamos `scipy.optimize.minimize(..., method="Powell")`, otro método derivative-free, pero mucho más sofisticado.

Random walk: resultado



Greedy (izq.): solo 10 de 500 pasos se aceptaron — encontró el pozo más cercano al punto de arranque y ya no pudo salir ($f = -0{,}54$, lejos del global $-3{,}29$). Puro (der.): se difunde sin rumbo, ni siquiera visita un pozo.

Random walk: una lección incómoda

¿Y qué hizo Powell (el método "profesional"de scipy)?

Arrancando del mismo punto, `scipy.optimize.minimize(..., method="Powell")` converge a $(0, 0)$ con $f \approx 0$ — ¡el punto de "trampa"débil entre pozos, ni siquiera un mínimo real de los 6 pozos! No encontró el mínimo global ($f = -3{,}29$).

- Ni el random walk greedy, ni Powell (un método local derivative-free serio), escapan solos de este paisaje con muchos mínimos locales.
- **Ningún optimizador puramente local** —con gradiente o sin él— tiene forma de "saber"que hay algo mejor del otro lado de una colina, si nunca la cruza.
- Por eso existen las **metaheurísticas** de la Clase 3: combinan un optimizador local con reinicios/aleatoriedad inteligente para tener más confianza de escapar de mínimos locales malos.

Nelder–Mead (amoeba) en 2D

Nelder–Mead: la idea

Idea

Mantiene un **simplex** (en 2D: un triángulo, 3 puntos) y lo va deformando para .ªrrastrarlo"hacia el mínimo — como una ameba que cambia de forma para moverse. Es derivative-free y suele ser mucho más eficiente que grid search o random walk.

Operaciones (ordenando los vértices: mejor B, medio, peor P; C=centroide sin P)

- **Reflexión**: $R = C + (C - P)$. Si $f(R)$ mejora al mejor, probar **expansión** $E = C + 2(R - C)$ (ir aún más lejos).
- Si R mejora al segundo peor (pero no al mejor): aceptar R.
- Si no mejora nada: **contracción** hacia adentro $C_t = C + 0{,}5(P - C)$.
- Si ni eso mejora: **encoger** todo el simplex hacia B.

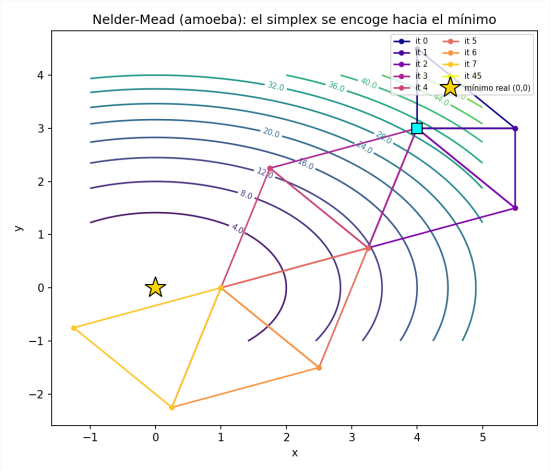
Nelder–Mead: ejemplo paso a paso

$f(x, y) = x^2 + 2y^2$ (la de la Clase 1), arrancando de $(4, 3)$:

```
it= 1  mejor=[4. 3.] f=34.00000   peor=[4.  4.5] f=56.50000   REFLEXION
it= 2  mejor=[4. 3.] f=34.00000   peor=[5.5 3. ] f=48.25000   EXPANSION
it= 3  mejor=[3.25 0.75] f=11.68750   peor=[5.5 1.5] f=34.75000   REFLEXION
it= 5  mejor=[1. 0.] f=1.00000   peor=[1.75 2.25] f=13.18750   REFLEXION
it= 8  mejor=[1. 0.] f=1.00000   peor=[ 0.25 -2.25] f=10.18750   CONTRACCION
      ...
it=45  ->  x* = [-1.7e-05, -2.6e-05]   f(x*) = 1.61e-09
```

El simplex se refleja, expande cuando va por buen camino, y se contrae/encoge cuando se pasa — sin usar ninguna derivada.

Nelder–Mead: evolución del simplex



Nelder–Mead: código

Manual (núcleo, un paso):

```
simplex.sort(key=f)
B, *resto, P = simplex
C = np.mean(simplex[:-1], axis=0)
R = C + (C - P)                          # reflexion
if f(R) < f(B):
    E = C + 2*(R - C)                    # expansion
    simplex[-1] = E if f(E)<f(R) else R
elif f(R) < f(resto[-1]):
    simplex[-1] = R
else:
    Ct = C + 0.5*(P - C)                 #
        contraccion
    if f(Ct) < f(P): simplex[-1] = Ct
    else: simplex = [B+0.5*(v-B) for v in
        simplex]
```

Con scipy:

```
from scipy.optimize import minimize

res = minimize(
    f, x0,
    method="Nelder-Mead")
```

Manual: 45 iteraciones, $f^* = 1{,}6 \times 10^{-9}$.
Scipy: 75 iteraciones, $f^* = 2{,}8 \times 10^{-17}$ (usa tolerancias internas más estrictas y ajustes finos que la versión didáctica de aquí).

Script completo: `scripts/06_nelder_mead_2d.py`

Síntesis

Comparación de los 5 métodos

	Bisección	**Áurea**	**Grid search**	**Random walk**	**Nelder–Mead**
Variables	1	1	1–3 (práctico)	cualquiera	cualquiera
Usa f'	sí	no	no	no	no
Determinista	sí	sí	sí	no (parcial)	sí
Garantiza mínimo (con supuestos)	local, si unimodal	local, si unimodal	de la malla (converge si $N \to \infty$)	no	local
Costo	$O(\log(1/\epsilon))$	$O(\log(1/\epsilon))$	$O(N^d)$	O(iter), ineficiente	O(iter), eficiente
Cuándo usarlo	f' barata, 1 variable	f costosa/no diferenciable	pocas variables, sin garantías sobre f	casi nunca solo (sirve de base para Clase 3)	muchas variables, sin derivadas, robusto

Lo que tienen en común

Todos comparten una limitación importante: son (o dependen fuertemente de ser) **métodos locales** o de **costo exponencial**. Ninguno de los cinco, tal como los vimos hoy, tiene una estrategia inteligente para escapar de un mínimo local malo en un paisaje con muchos pozos, sin pagar el precio exponencial del grid search.

La pregunta que abre la Clase 3

¿Cómo combinamos la eficiencia de un optimizador local (como Nelder–Mead) con la capacidad de **escapar** de mínimos locales (como random walk, pero de forma más inteligente)?

Vista previa: Clase 3

Clase 3: metaheurísticas

- **Basin hopping**: alterna reoptimización local (como Nelder–Mead) con saltos aleatorios (como random walk) y un criterio de aceptación tipo Metrópolis.
- **Simulated annealing**: el mismo criterio de Metrópolis, pero con una "temperatura"que baja con el tiempo — mucha exploración al inicio, mucha explotación al final.
- **Algoritmos genéticos**: una **población** de soluciones que evoluciona por selección, cruce y mutación.
- **Particle swarm optimization**: un **enjambre** de partículas que se mueven influenciadas por su mejor posición y la del grupo.

Todos, otra vez, con código manual **y** su equivalente en scipy (donde exista) para comparar.

Ejercicios propuestos

Ejercicios para la próxima clase

1. Use bisección para minimizar $f(x) = x^4 - 3x^2 + x$ en un intervalo donde f' cambie de signo. Verifique con `scipy.optimize.brentq`.
2. Compare, para tres tolerancias distintas ($10^{-3}, 10^{-6}, 10^{-9}$), cuántas iteraciones necesitan bisección y búsqueda áurea. ¿La diferencia crece o se mantiene constante?
3. Implemente grid search en 4 variables para $f(x_1, x_2, x_3, x_4) = \sum x_i^2$ con $N = 20$ puntos por eje. Mida el tiempo de ejecución y compárelo con el caso $N = 20, d = 2$.
4. Modifique `scripts/05_random_walk_2d.py` agregando un *reinicio*: si tras 50 pasos no hay mejora, saltar a un punto aleatorio nuevo. Compare el resultado contra la versión sin reinicios.
5. Modifique `scripts/06_nelder_mead_2d.py` para $f(x, y) = (x-2)^2 + (y+1)^2 + 5\sin(x)\cos(y)$ (no convexa) y observe si el simplex encuentra el mínimo global o queda atrapado.
6. Para el paisaje de 6 pozos (`scripts/_landscape.py`), corra grid search 2D con $N = 10, 25, 50, 100$ y grafique el error respecto al mínimo verdadero vs. el número de evaluaciones.
7. **Reflexión:** explique con sus palabras por qué Nelder–Mead normalmente necesita muchas menos evaluaciones que grid search para la misma precisión, y en qué casos esa ventaja desaparece.

Referencias

Referencias

- J. Nocedal, S. Wright. *Numerical Optimization*, 2.ª ed. Springer, 2006.
- M. Kochenderfer, T. Wheeler. *Algorithms for Optimization*. MIT Press, 2019. (Cubre bisección, áurea, Nelder–Mead y búsquedas estocásticas con código.)
- W. Press, S. Teukolsky, W. Vetterling, B. Flannery. *Numerical Recipes*, 3.ª ed. Cambridge University Press, 2007. (Capítulo 10: minimización 1D y multidimensional.)
- J. Nelder, R. Mead. .ᴀ Simplex Method for Function Minimization."*Computer Journal*, 7(4), 1965.
- D. Boyd, J. Vandenberghe. *Convex Optimization*. Cambridge University Press, 2004.
- Documentación de `SciPy` (`scipy.optimize`): `https://scipy.org`.

¿Preguntas?